Individual reflection: naive search and KMP

For this assignment, I worked individually and was responsible for understanding both naive substring search and KMP. Although both algorithms solve the same problem, comparing them showed me how much the structure of an input can affect the amount of work an algorithm performs.

Worst cases

The worst case for naive search occurs when the pattern almost matches at many different positions. For example, if the text consists entirely of a characters and the pattern consists of a long sequence of a characters followed by b, the search gets almost to the end of the pattern before failing. It then moves forward one position and repeats many of the same comparisons. This leads to a worst-case time complexity of O(nm), where n is the text length and m is the pattern length.

KMP handles that repeated-input case much better because it uses its LPS table to avoid starting over after every mismatch. Its worst-case time complexity is O(n + m), including the time needed to build the table. It can still make more than one comparison for a text character when it falls back, but it does not move backwards through the text.

A design decision

One important design decision was whether to include the time needed to build KMP's LPS table in the benchmark. I included it in the comparison because the experiment measures a complete, one-off search. If someone searches a text once, KMP needs to prepare the pattern before it can search, so leaving that work out would give an incomplete picture.

This decision affected the results. When the pattern matched at the beginning, naive search could return immediately after checking it. KMP had to build its table first, making it slower in that case. If the same pattern were used to search many texts, building the table once and reusing it would be a different and worthwhile experiment.

What I would change

If I started again, I would keep the one-off comparison but also add a separate experiment that searches multiple texts using the same pattern. This would show how

much difference it makes to spread KMP's preparation cost across several searches. I would keep the two experiments separate so that it is clear what each timing includes.

I would also test a wider range of ordinary text. The current prose results use related sections of one excerpt, so more independent documents would help show whether the close results between the algorithms occur in other situations.

What I understand better

KMP's fallback is the part I would focus on when explaining what I learned. In the example ABABABAC searched for ABABAC, the first five characters match before a mismatch occurs. KMP can keep ABA as part of a possible new match because it is both the end of the matched text and the beginning of the pattern. It then continues without moving backwards through the text and finds the match at position 2.

This helped explain why KMP needs the LPS table: it does not simply skip characters at random. The table records which parts of an earlier match are still useful. Comparing this with naive search also made the practical trade-off clearer: KMP avoids repeated work, but its extra preparation is not always worthwhile for a search that finishes quickly.